Lucien Roberson
HW 4
Oct 9

Chapter 13:

Q1)
From the “man” command of “free” I can see that free will show the physical and swap memory in the system.
Q2)

```
vboxuser@Lucien:~/5600/hw4$ free
               total        used        free      shared  buff/cache   available
Mem:         4005808     2191904      229120       73708     1668160     1813904
Swap:              0           0           0
```

Default unit of “free” is a kibibyte, 1024 bytes.
4,005,808 kibibytes matches up to allotted 4096MB I gave this VM.
I thought that only 229,120KiB looked really low but I read that Linux uses free memory for buffers and cache

Q3)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(int argc, char *argv[]) {
    size_t n = (size_t)atoi(argv[1]) * 1024 * 1024;
    int seconds = (argc > 2) ? atoi(argv[2]) : 0;

    char *arr = malloc(n);

    time_t start = time(NULL);
    while (seconds == 0 || time(NULL) - start < seconds) {
        for (size_t i = 0; i < n; i++)
            arr[i]++;
    }

    free(arr);
    return 0;
}
```

Q4)

```
vboxuser@Lucien:~$ free --mega
               total        used        free      shared  buff/cache   available
Mem:            4101        1772         192          55        2320        2328
Swap:              0           0           0
vboxuser@Lucien:~$ free --mega
               total        used        free      shared  buff/cache   available
Mem:            4101        1868         298          55        2124        2233
Swap:              0           0           0
```

The above screenshot is the first and second “free –mega” calls before and while the command “$ ./memory-user 100” were called.

```
vboxuser@Lucien:~$ free --mega
               total        used        free      shared  buff/cache   available
Mem:            4101        2724         360          63        1254        1377
Swap:              0           0           0
vboxuser@Lucien:~$ free --mega
               total        used        free      shared  buff/cache   available
Mem:            4101        3707         167          63         487         394
Swap:              0           0           0
```

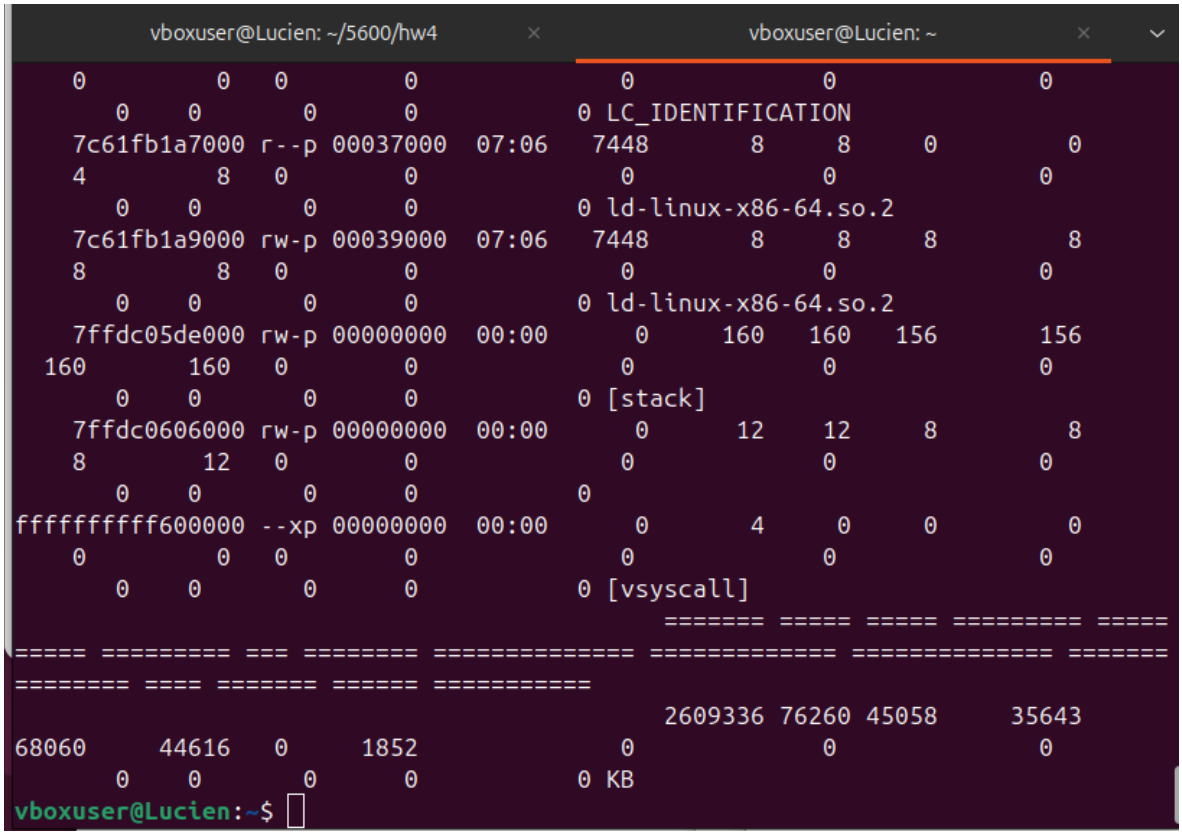
The above screenshot is the same but with 1000MB instead of 100.
Neither is perfect and if anything I would expect a little bit more memory to be used but in both cases the amount of used memory is less than the amount memory-user called for (96 vs 100 and 978 vs 1000)

Q5) Reading pmap man. Shows a memory map.
Q6) I ran pmap on a firefox process because I thought the note looked interesting “-isForBro”

```
vboxuser    2808  0.0  1.5 2606260 61532 ?       Sl   22:10   0:01 /snap/firefox
/7766/usr/lib/firefox/firefox -contentproc -isForBro
```

I only then realized that “Bro” was just the word “Browser” that had been shortened.

Q7) Running “pmap 2808 -X” showed that there was a massive range of entities from vvar to vvar_clock, vdso, anon:jemalloc, and many others along with stack and heap. \



Q8)
I ran memory-user twice with args of 100 and 1000MB and saw that all of that memory is in Heap

This is the first call

```
        0    0       0      0           0 [heap]
    706ea09ff000 rw-p 00000000  00:00       0 102404 102404 102404    102404
102404    102404   0        0              0             0              0
        0    0       0      0           0
```

and this is the second

```
          0    0       0      0           0 [heap]
    7f4b343ff000 rw-p 00000000  00:00       0 1024004 1024004 1024004   1024004
  1024004   1024004   0        0              0             0              0
          0    0       0      0           0
```

They match up perfectly as expected.

Chapter 14:

Q1)

```c
#include <stddef.h>

int main() {
    int *p = NULL;
    *p = 0;
    return 0;
}
```

```
vboxuser@Lucien:~/5600/hw4$ gcc null.c -o null
vboxuser@Lucien:~/5600/hw4$ ./null
Segmentation fault (core dumped)
```

Q2)
The debugger found the error

```
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555513d in main () at null.c:5
5           *p = 0;
```

Q3)
I got the following valgrind output:

“==7855== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
==7855==
==7855== 1 errors in context 1 of 1:
==7855== Invalid write of size 4
==7855== at 0x10913D: main (null.c:5)
==7855== Address 0x0 is not stack'd, malloc'd or (recently) free'd
==7855==
==7855== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
Segmentation fault (core dumped)”
It says that it is an invalid access.

Q4)
I made the program leak.c

```c
#include <stdlib.h>

int main() {
    int *p = malloc(sizeof(int));
    *p = 0;
    return 0;
}
```

I did not find any errors in gdb but I did find them with valgrind.

==8148== LEAK SUMMARY:
==8148== definitely lost: 4 bytes in 1 blocks
==8148== indirectly lost: 0 bytes in 0 blocks
==8148== possibly lost: 0 bytes in 0 blocks
==8148== still reachable: 0 bytes in 0 blocks
==8148== suppressed: 0 bytes in 0 blocks
==8148==
==8148== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

Q5)

```c
#include <stdlib.h>

int main() {
    int *data = malloc(100 * sizeof(int
    data[100] = 0;
    free(data);
    return 0;
}
```

My program showed this error in valgrind:
“==8325== Invalid write of size 4
==8325== at 0x10918D: main (overflow.c:5)
==8325== Address 0x4a7e1d0 is 0 bytes after a block of size 400 alloc'd
==8325== at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==8325== by 0x10917E: main (overflow.c:4)”

The program has a buffer in heap, but it won’t throw an error, if you tried to grab the data in that overflow you’d probably get a random value from a previous process.

Q6)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *data = malloc(100 * sizeof(int));
    data[0] = 5;
    free(data);
    printf("%d\n", data[0]);
    return 0;
}
```

It doesn’t throw an error but it returns a random value. Valgrind reports the following error
“==8840== 1 errors in context 1 of 1:
==8840== Invalid read of size 4
==8840== at 0x1091BD: main (useafterfree.c:8)
==8840== Address 0x4a7e040 is 0 bytes inside a block of size 400 free'd
==8840== at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==8840== by 0x1091B8: main (useafterfree.c:7)
==8840== Block was alloc'd at
==8840== at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==8840== by 0x10919E: main (useafterfree.c:5)”

It’s trying to read from memory that has already been freed, that’s why it’s producing gibberish.

Q7)
I made a program called badfree.c

It gave an error right off the bat with this output “badfree.c: In function ‘main’:
badfree.c:5:5: warning: ‘free’ called on pointer ‘data’ with nonzero offset 200 [-Wfree-nonheap-object]
  5 | free(data + 50);
  | ^~~~~~~~~~~~~~~
badfree.c:4:17: note: returned from ‘malloc’
  4 | int *data = malloc(100 * sizeof(int));
  | ^~~~~~~~~~~~~~~~~~~~~~~~~~~”

Valgrind shows this issue more thoroughly
“==9169== Invalid free() / delete / delete[] / realloc()
==9169== at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==9169== by 0x109194: main (badfree.c:5)
==9169== Address 0x4a7e108 is 200 bytes inside a block of size 400 alloc'd
==9169== at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==9169== by 0x10917E: main (badfree.c:4)”

Passing a pointer from the middle of the array to free makes the program crash.

Q8)

```c
#include <stdio.h>
#include <stdlib.h>

int *data = NULL;   // the array
int size = 0;       // number of elements

void add(int x) {
    data = realloc(data, (size + 1) * sizeof(int));
    data[size] = x;
    size++;
}

int main() {
    for (int i = 0; i < 10; i++)
        add(i);

    for (int i = 0; i < size; i++)
        printf("%d ", data[i]);
    printf("\n");

    free(data);
    return 0;
}
```